

From Vibe Coding to Spec-Driven Agentic Engineering

A Hands-On Implementation using Claude Code

PART IX

Instructor: Muhammad Arif Butt, PhD

About The Instructor



Dr. Muhammad Arif Butt
Asst. Prof. Department of Data Science
University of Punjab, Lahore
arif@pucit.edu.pk
https://arif.excaliat@gmail.com
https://arifbutt.me
https://www.linkedin.com/in/dr-arif-butt/
https://youtube.com/learnwitharif
https://github.com/arifpucit
https://huggingface.co/arif-butt

- **Education:**
  - Graduation from Pakistan Military Academy, Kakul
  - MPhil CS from University of Punjab, Lahore
  - PhD CS from University of Punjab, Lahore
- **Experience:**
  - Served in field/staff/instructional posts in Pakistan Army
  - Assistant Professor, Department of Data Science
  - Founder Excaliat (https://excaliat.com/en)
  - Founder FalconHunt (https://falconhunt.org/)
  - Co-Founder Tbox Solutionz (https://tboxsolutionz.com/)
- **Research Interest:**
  - Embedded and Real-Time Operating Systems
  - Vulnerability Analysis, Binary Exploitation & Exploit Development
  - AI-Driven Cybersecurity and Securing AI systems

Scheme of Sessions

Part - I: Overview of Agentic Coding Assistants

Part – II: Claude Ecosystem, Installing & using Claude Code

Part – III: Slash Commands, Context Window & `.claude` Directory

Part – IV: Workflow of Building a Basic Vulnerable Web Application

Part – V: Exploiting Vulnerable Web Application

Part – VI: Workflow of Implementing Features and Fixing Bugs

Part – VII: Collaborative Workflow: Claude Code + GitHub + Jira

Part – VIII: Built-in Skills & Vuln-Web-App Current Status

Part – IX: **Creating & using Custom Skills & Skill Marketplace**

Important Concepts Related to SKILLS

Instructor: Muhammad Arif Butt, PhD

Recap: Four Kinds of / in Claude Code

Built-in slash commands
Fixed actions baked into the CLI

Control the session itself — switch models, manage context, sign in. The CLI runs them directly; the model isn't involved.

WHERE

`No files — compiled into Claude Code`

`/help` `/model` `/clear` `/config` `/context` `/compact` `/login`

You run them — Claude can't trigger them. 60+ ship with Claude Code; list them with `/help`.

Custom slash commands
A reusable prompt you write

A Markdown file whose contents become a prompt template, invoked by typing `/name`. Supports `$ARGUMENTS`. The older, simpler format.

WHERE

`.claude/commands/<name>.md (project)`
`~/.claude/commands/<name>.md (personal)`

`/deploy` `/fix-issue` `/optimize`

Manual only — runs just when you type it. Legacy format, still supported.

Built-in skills
Skills that ship with Claude

Each is an instruction set Claude runs with its tools. Invoke with `/name` — or Claude fires it itself when your request matches.

WHERE

`Bundled with Claude Code · marked Skill in /commands`

`/code-review` `/debug` `/commit` `/pdf`

The short description stays in context so Claude knows it exists; the full body loads only on use. ~10 ship with Claude Code.

Custom skills
A SKILL.md folder you write

YAML frontmatter + steps, plus optional scripts and files. Invoke with `/name`, or let Claude load it automatically when work matches.

WHERE

`.claude/skills/<name>/SKILL.md (project)`
`~/.claude/skills/<name>/SKILL.md (personal)`

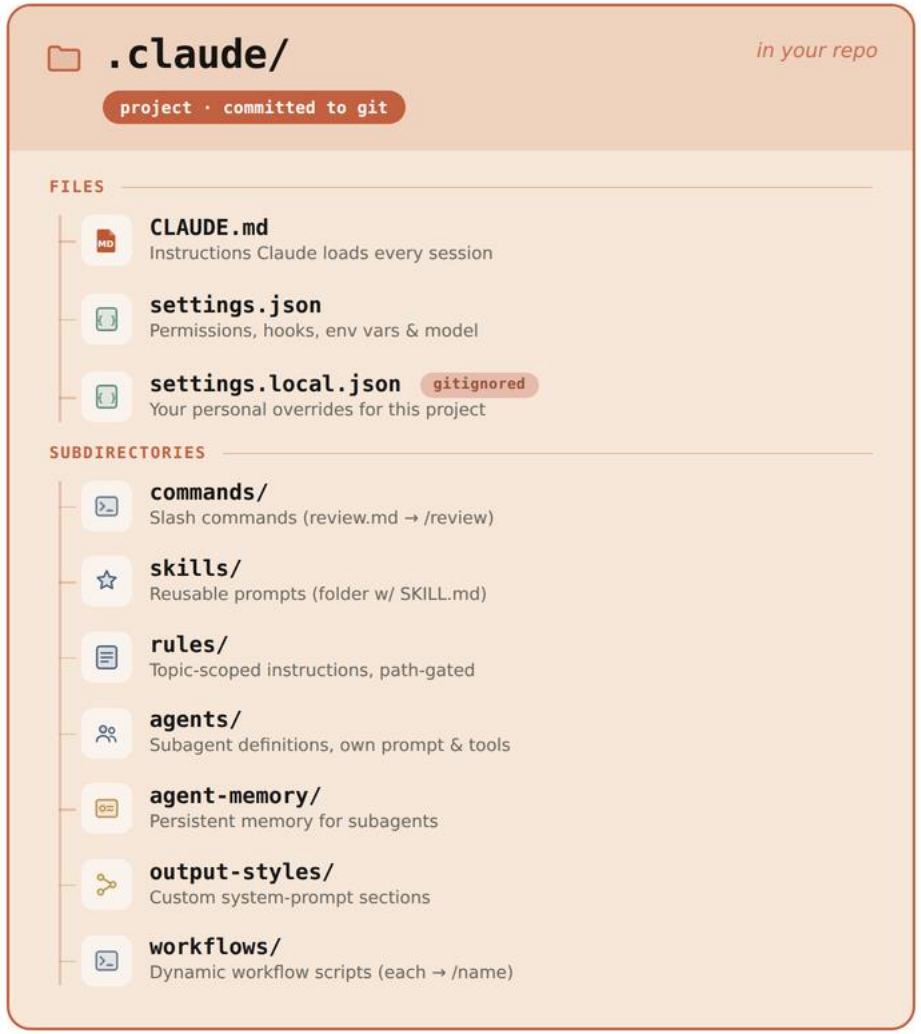
`/summarize-changes` `/migrate-component`

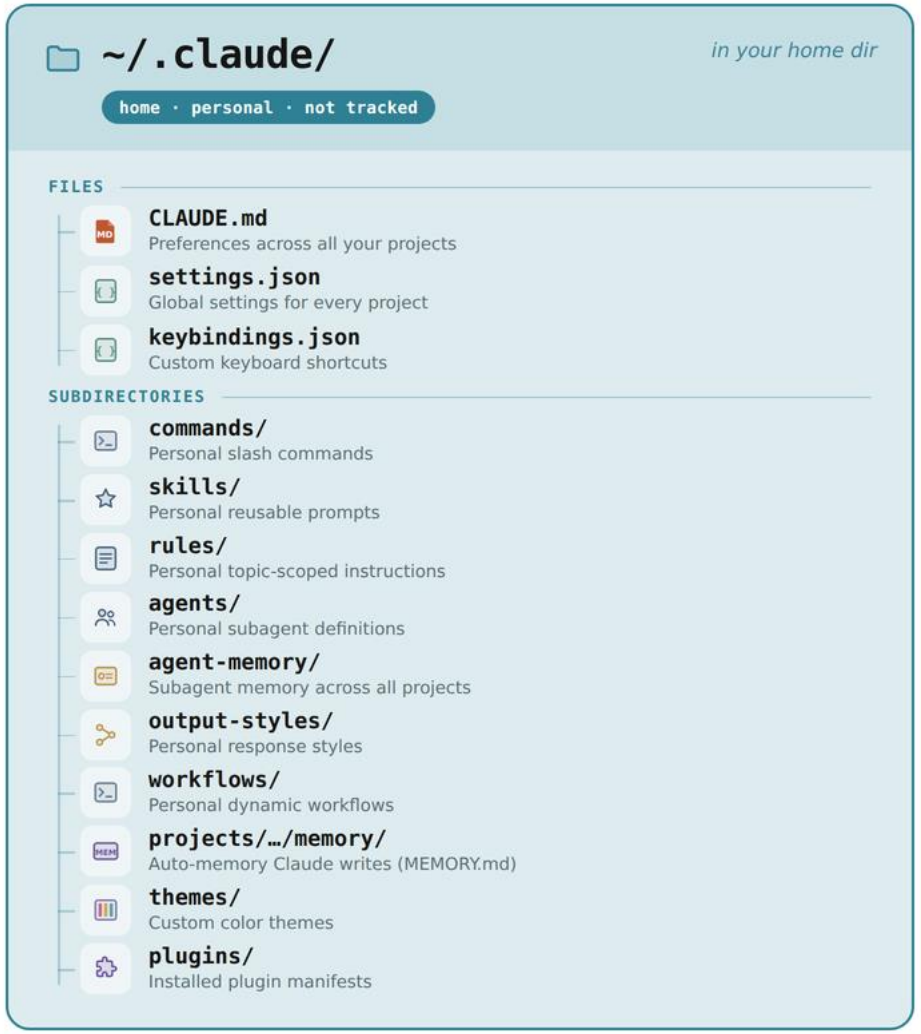
Recommended format (v2.1.101+). Legacy `/commands` still work; if names clash, the skill wins.

- **Core Difference:** A custom slash command is invoke-only: nothing happens until you type /name. A skill adds an always-loaded description, so Claude can also trigger it on its own when the task matches — and it can bundle scripts and supporting files.
- **Why Skills are now Preferred:** Skills are a superset of commands — manual /name plus autonomous use — with progressive disclosure: only the short description sits in context, the body loads on demand. One unified system replaces two.

Recap: The `.claude/` Directory

The .claude directory is the central control centre and memory store for Claude Code, and it stores project-specific guidelines, custom tools, permission settings, and session history





Custom Slash Commands in Claude Code

1 · Create a file in .claude/commands/

my-project/
.claude/
commands/
git-workflow.md
encodes branching, commit & PR conventions
explain.md
explains a file or function in plain language
deep-security-audit.md

2 · The file is the command's prompt

deep-security-audit.md

- What to scan
- How to scan
- Tools to use
- Output format
- ...

Exit Session

3 · Type the slash command to run it

/deep-security-audit

Scope — where you put the file decides who can run it

Project Scope

.claude/commands/

origin/main

- Version controlled
- Shared through Git
- Team-wide behavior

Personal Scope

~/.claude/commands/

- Local machine
- Available in all projects
- Personal productivity shortcuts

```
/
review-checklist
fix-issue
pr-review
```

Custom Skills in Claude Code

1 A skill is a folder on disk — where it lives & what's inside

📁 SKILL DIRECTORY STRUCTURE

```
.claude/skills/
└─ deep-security-audit/
   ├─ SKILL.md            • required
   ├─ scripts/            # .py / .sh — Claude runs
   │   ├─ dep_scan.sh
   │   └─ process_data.py
   ├─ references/         # docs, read on demand
   │   └─ vulnerability-checklist.md
   └─ assets/             # templates · fonts · images
       └─ report-template.md
```

💡 The folder name must match the **name:** field in SKILL.md **exactly** (case-sensitive).

• required ○ optional — add only what the skill needs

SKILL.md — REQUIRED

The one file every skill must have. Holds the **YAML frontmatter** (`name` + `description`) plus the **instructions / workflow** Claude follows.

CONTAINS: metadata (name + description) · instructions / workflow body

scripts/ — OPTIONAL

Executable code Claude runs via Bash — ideal for deterministic steps (scanners, validators, converters). Run, **not** read into context.

e.g. .py · .sh · .js · .rb · any language

references/ — OPTIONAL

Docs loaded only on demand — read just-in-time, only when a step calls for them. Keeps SKILL.md small while allowing deep detail.

e.g. checklists · specs & standards · API guides · schemas

assets/ — OPTIONAL

Files used in the output (not instructions). Claude fills or copies them to build the final result — e.g. a `report-template.md`.

e.g. templates · images / logos · fonts · sample data

Custom Skills in Claude Code (cont...)

2 Inside SKILL.md — frontmatter + body, and what loads when

SKILL.md

```
1 ---
  name: deep-security-audit
  description: >
    Run a security audit. Use when a user
    asks to audit / pentest / find vulns …
2 arguments: [target]
  allowed-tools: Read, Grep, Bash(git clone:*)
  model: inherit
  context: fork
  agent: Explore
  hooks: { PreToolUse: ./guard.sh }
  shell: bash
  disable-model-invocation: true
  ---
  # Deep Security Audit
  Target: $ARGUMENTS  (path | git-url)
  No arg → audit current dir.
  Map the app, run scanners, then review
  by OWASP Top 10 (cite file:line).
  Write security-audit-report.md
3 read references/vulnerability-checklist.md
  run  scripts/dep_scan.sh
```

● ① always loaded ● ② on trigger ● ③ on demand

{ } YAML frontmatter (metadata, at the top)

name REQUIRED ≤ 64 chars · lowercase-hyphen · **must match the folder name**.

description REQUIRED ≤ 1024 chars · says **what it does + WHEN to use** → this drives auto-discovery.

OPTIONAL

`arguments` `allowed-tools` `model` `context` `agent` `hooks` `shell` `disable-model-invocation` `version` `metadata`

💡 The **description** is the #1 thing that makes Claude pick the skill — be explicit about **WHEN** to use it, not just what it does.

Markdown body (< 500 lines / 5k tokens)

- **Instructions** — the step-by-step workflow Claude follows.
- **Patterns** — recommended approaches & best practices.
- **Pitfalls** — common mistakes and how to avoid them.
- **Validation steps** — how to check the output (may run scripts).
- **References** — when to read references/ & run scripts/.

Progressive disclosure — only what's needed loads

1. **At startup** — name + description only — always in context
2. **On trigger** — full SKILL.md body — < 500 lines / 5k tokens
3. **On demand** — referenced files & scripts — read / run just-in-time

Custom Skills in Claude Code (cont...)

3 Invocation & scope — who runs the skill, and where it lives

WHO RUNS IT · disable-model-invocation

disable-model-invocation: false — DEFAULT

Claude —— Claude decides ——→ SKILL.md

You —— /skill ——→ SKILL.md

Claude can invoke it **automatically** — and you can still run it manually. **Default** · right for most skills.

disable-model-invocation: true — MANUAL ONLY

You —— /skill ——→ SKILL.md

Claude ---- auto-invoke blocked ----→ blocked

Manual only — Claude won't auto-run it; you must type `/skill`. Best for side-effects: deploys, migrations, anything destructive.

WHERE IT LIVES · SCOPE TIERS

Project

`.claude/skills/`

Committed to the repo — shared with the whole team.

USE FOR team conventions, repo-specific workflows

Personal

`~/.claude/skills/`

On your machine — available in all your projects.

USE FOR your own helpers, across every project

Plugin

```
~/.claude/plugins/cache/
<marketplace>/<plugin>/<version>/
skills/<skill-name>/SKILL.md
```

Bundled & shipped inside a plugin you install.

USE FOR distributable, versioned, updatable skills

Name clash? A **project**-level skill takes priority over a **personal** (global) one with the same name.

Custom Skills in Claude Code (cont...)

4 How a skill runs — lifecycle, security & limits

RUNTIME LIFECYCLE · what happens, step by step

1. **Advertise at startup**
   Every skill's **name + description** is preloaded into the system prompt. Claude knows it exists — nothing else loads yet.
2. **Trigger / match**
   Your request (or naming it) matches the **description** → the skill activates. User-invoked **or** model-invoked.
3. **Load instructions**
   Claude reads the **full SKILL.md body** into context.
4. **Resolve the argument**
   Request + `$ARGUMENTS` become the task — a path or git-url; none → current directory.
5. **Pull referenced files**
   `references/*` are read in only where the body says to — **just-in-time**.
6. **Run scripts**
   `scripts/*` are executed via Bash — they run **without being read** into context.
7. **Execute & output**
   Claude follows the steps using only **allowed-tools** and produces the result — e.g. `security-audit-report.md`.
8. **Persists across turns**
   Recently-used skills stay attached within a **token budget** — no manual reload, keeping context relevant while saving tokens.

Security — skills can run code

A skill may **execute scripts and touch your files, env vars & secrets**. Install only from **trusted sources**; audit `SKILL.md`, `scripts/` & `references/` before enabling.

allowed-tools = least privilege. Grant the minimum the skill needs — it can never call a tool you didn't list (your sandbox boundary).

Size & structure limits

- `name` ≤ 64 chars (matches folder) · `description` ≤ 1024
- SKILL.md body **< 500 lines / < 5k tokens**
- keep `references/` one level deep
- use relative, forward-slash paths (agent-readable)

Skill vs slash command · portable

SLASH COMMAND	SKILL
user-typed only · single `.md` · `$ARGUMENTS`	user **or** auto-invoked · bundles scripts/refs · 3-level loading

🌐 **Open standard** (agentskills.io) — the same `SKILL.md` runs across many agents.

Authoring your Own Skills

Instructor: Muhammad Arif Butt, PhD

Four Approaches of Authoring a Skill

◀ more control · more effort

faster · less writing ▶

1 — 2 — 3 — 4

1 Manual authoring

HOW IT WORKS

You write SKILL.md by hand — define the YAML frontmatter (name, description) and the instructions Claude follows.

EXAMPLE

```
---
name: my-skill
description: what + when
---
# Instructions …
```

PROS

- ✓ Full control of wording & structure
- ✓ No external dependencies
- ✓ Deepest grasp of the format

WATCH-OUTS

- Slowest of the four
- Must know the expected structure

BEST FOR

bespoke, precise skills

Control ●●●●
Speed ●○○○

2 Download / tailor skill from GitHub

HOW IT WORKS

Find a ready-made skill, download its folder from the **GitHub repo**, then tailor the SKILL.md to your own conventions.

STEPS

```
# 1 find a ready-made skill
# 2 open its GitHub repo
# 3 download the skill folder
# 4 drop into .claude/skills/
# 5 tailor SKILL.md to fit
> restart & use it
```

PROS

- ✓ Starts from a proven, working skill
- ✓ Good control + speed balance

WATCH-OUTS

- Inspect the SKILL.md before trusting it
- Tool can be flaky — clone as a fallback

BEST FOR

adapting an existing skill

Control ●●●○
Speed ●●○○

3 AI-assisted generation

HOW IT WORKS

Ask Claude (the **skill-creator** skill) to build it interactively — it interviews you, drafts the SKILL.md, and sets up tests.

EXAMPLE

```
"Use skill-creator to
build a skill that…"
# interview → draft SKILL.md
# + evals, then
# package_skill.py
```

PROS

- ✓ Fast — great for beginners
- ✓ Claude writes the boilerplate
- ✓ Built-in test & eval loop

WATCH-OUTS

- Review output for accuracy & fit
- Less control over details

BEST FOR

quick starts & learning

Control ●●○○
Speed ●●●○

4 Marketplace / plugin install

HOW IT WORKS

Browse Claude Code's marketplace with the **/plugin** command and install a skill someone else already built.

EXAMPLE

```
/plugin → Discover → install
# native to Claude Code
/plugin marketplace add
anthropics/claude-plugins-official
# 3rd-party catalog:
npx <catalog-cli> add <skill>
```

PROS

- ✓ Fastest — no writing at all
- ✓ Official plugins are vetted by Anthropic

WATCH-OUTS

- Relying on someone else's instructions
- Audit external sources — especially **npx**

BEST FOR

common, already-solved tasks

Control ●○○○
Speed ●●●●

Download/Tailor Skill from GitHub

1 Find a skill

Search **skillsmp.com** for the kind of skill your task needs.

skillsmp.com

```
Search: "the skill you need"

▸ skill-name by author ★ 1.2k
matches your task
— browse results, open the one you want
```

2 Copy its GitHub URL

Open the skill's **GitHub page** (linked on skillsmp) and copy the folder URL.

github.com

```
github.com/owner/repo
/tree/main/skills/
skill-name/ [ copy ]
SKILL.md README …
◉ skim SKILL.md before you trust it
```

3 Download the folder

Paste the URL into **download-directory.github.io** → get a .zip.

download-directory.github.io

```
url ▸ .../skills/skill-name

✓ Downloaded 3 files! Done!
zipping → skill-name.zip
extract it — macOS: run unzip in Terminal
```

4 Drop it in your project

Create **.claude/skills/<name>/** in your project root; put SKILL.md inside.

my-project

```
my-project/
└ .claude/
└ skills/
└ skill-name/ └ SKILL.md
◉ SKILL.md sits directly inside — don't double-nest
```

5 Restart & use it

Restart Claude Code so it discovers the skill, then ask in plain language.

Terminal

```
$ claude
> do the task in plain words
• Using skill-name skill…
✓ done — skill applied
no need to name it — Claude auto-detects
```

Tips & fallbacks

- **Inspect first.** Read the SKILL.md — these are third-party skills.
- **No unzip tool?** Clone the repo: `git clone --depth 1 <repo-url>`
- **macOS stuck?** In Terminal run `unzip <skill>.zip` (skips Archive Utility)
- **Keep it flat.** One skill = one folder directly inside `.claude/skills/`

Summary

the workflow, end to end

1 Find skill → 2 Copy URL → 3 Download .zip → 4 Install → 5 Restart & use

Hands on Practice: Download Skill from GitHub



https://skillsmp.com/

Using `/skill-creator` Skill

Manual Download vs Plugin Install

Manual download
Find it, then drop it in yourself

Find a skill on skillsmp.com or GitHub, download the folder, and place it in your skills directory.

Where

```
~/.claude/skills/<name>/ (personal)
.claude/skills/<name>/ (project)
```

`skillsmp.com` `GitHub`

`Manual updates`

Best when you want to read or tweak it.

https://skillsmp.com/

/plugin install
Install from a marketplace

Installs the skill inside a plugin: a package with one skill or a whole bundle (may add agents, hooks, MCP).

Where

```
~/.claude/plugins/ (shared cache)
```

`Auto-updates` `Every project`

`One skill or a bundle`

One unit; versions and updates handled.

Skills vs Plugins vs Marketplaces

◀ smallest unit · one capability — a plugin bundles skills ▶ — a marketplace lists plugins ▶ whole catalog · many plugins ▶

1 — 2 — 3

LEVEL 1 Skill

WHAT IT IS

One capability. A `SKILL.md` — name, description, and instructions — optionally with bundled scripts, references, and assets.

HOW IT RUNS

Loads on demand. Claude triggers it automatically, or you call it with `/skill-name`.

WHERE IT LIVES

`.claude/skills/<name>/` in a project, or `~/.claude/skills/` for all projects.

EXAMPLES

brand-guidelines · pdf · skill-creator

LEVEL 2 Plugin

WHAT IT IS

A bundle. Packages one or more skills together — plus optional commands, sub-agents, hooks, and MCP servers.

HOW YOU GET IT

Install as one unit:

```
/plugin install <plugin>@<marketplace>
```

WHERE IT LIVES

`~/.claude/plugins/cache/` once installed from a marketplace, then enabled in `.claude/settings.json`.

EXAMPLE

skill-creator = the skill + 4 agents (Executor, Grader, Comparator, Analyzer) + scripts

LEVEL 3 Marketplace

WHAT IT IS

A registry (a GitHub repo) that lists many plugins. You add it once, then install plugins from it.

HOW YOU GET IT

```
/plugin marketplace add <owner/repo>
```

— then browse with `/plugin`.

WHAT IT HOLDS

Many plugins, each with their own skills, agents, and tools.

EXAMPLES

claude-plugins-official (built into Claude Code) · **anthropics/skills**

Skills Marketplace

OFFICIAL maintained by Anthropic

claude-plugins-official

`anthropics/claude-plugins-official`

BUILT-IN

CONTAINS
Production-grade plugins — skill-creator, GitHub integration, LSP code-intelligence, and more.

TRUST
Hand-curated by Anthropic — inclusion at Anthropic's discretion. Highest trust.

HOW YOU GET IT
Built in — nothing to add. Install via `<name>@claude-plugins-official`

https://github.com/anthropics/claude-plugins-official

anthropics/skills

`github.com/anthropics/skills`

ADD MANUALLY

CONTAINS
Anthropic's example skills — document-skills (pdf, docx, pptx, xlsx) + example-skills demos.

TRUST
By Anthropic, as reference material — a learning shelf, not a production catalog.

HOW YOU GET IT
Add manually: `/plugin marketplace add anthropics/skills` → registers as `anthropic-agent-skills`

https://github.com/anthropics/skills

COMMUNITY built by the ecosystem

claude-community

`anthropics/claude-plugins-community`

VETTED

CONTAINS
Third-party plugins the public submits via claude.ai.

TRUST
Passed Anthropic's automated security screening — each plugin pinned to a commit SHA.

HOW YOU GET IT
Add manually: `/plugin marketplace add anthropics/claude-plugins-community` → installs as `@claude-community`

https://github.com/anthropics/claude-plugins-community

SkillsMP

`skillsmp.com`

AUDIT FIRST

CONTAINS
Independent search engine indexing ~1.8M public GitHub skills.

TRUST
No Anthropic vetting — light filtering only. Widest choice, lowest assurance.

HOW YOU GET IT
Not a plugin marketplace. Browse `skillsmp.com`, then copy the skill into `.claude/skills/` yourself.

https://skillsmp.com/

Installing & Invoking /skill-creator Skill

1 Add the marketplace

Register Anthropic's official catalog. **claude-plugins-official** ships with Claude Code — this just **makes sure the catalog is there**; nothing is installed yet.

```
/plugin marketplace add \
  anthropics/claude-plugins-official
✓ marketplace added
  catalog ready — 0 plugins installed yet
```

2 Install the plugin

Install **skill-creator** from that catalog, then **reload** to activate its skill in the current session.

```
/plugin install
  skill-creator@claude-plugins-official
/reload-plugins
✓ installed & activated
```

3 Invoke skill-creator

Ask in plain language (or type **/skill-creator**), then pick a mode — Create, Eval, Improve, or Benchmark.

```
>_ Terminal
$ claude
> Create a skill that reviews
  PRs for security issues.
● Using skill-creator…
● Mode: Create — gathering specs…
```

4 What it gives you

Four guided **modes** cover the whole skill lifecycle, each powered by four composable **agents**.

SKILL-CREATOR — MODES + AGENTS

MODES	AGENTS
Create new skill from scratch	**Executor** runs the skill
Eval run test prompts	**Grader** scores outputs
Improve iterate on feedback	**Comparator** blind A/B test
Benchmark variance analysis	**Analyzer** suggests fixes

5 Package the skill

When you're happy with it, hand it back to skill-creator to **package** it into a portable **.skill** file.

```
> Package my skill so I can
  install and share it.
● Using skill-creator…
● python -m scripts.package_skill…
✓ pr-security-review.skill created!
```

6 Result

A portable, tested skill — ready to drop into any project or share with your team.

SKILL

Packaged Skill

SKILL.md · scripts · references

tested · **evaluated** · optimized

Example: **pr-security-review.skill**

💡 Summary — skill-creator Workflow

1 **Add marketplace** → 2 **Install plugin** → 3 **Invoke /skill-creator** → 4 **Pick a mode** → 5 **Package .skill** → 6 **Share & install**

Hands on Practice: Installing & Invoking `/skill-creator`



OWASP Agentic Skills Top 10

01 AST01: Malicious Skills
Some skills are built to cause harm—stealing data, running hidden commands, or taking actions you never approved.

02 AST02: Supply Chain Compromise
Hacked dependencies, registries, or download channels can swap in a tampered skill—what you install isn't what was shipped.

03 AST03: Over-Privileged Skills
A skill is granted more access than it needs, so if it's ever compromised the potential damage is far larger.

04 AST04: Insecure Metadata
Missing, inaccurate, or faked metadata makes a skill look safer or different from what it actually does.

05 AST05: Unsafe Deserialization
Loading a skill's config files (YAML/JSON) without safe parsing lets hidden payloads run code on your system.

06 AST06: Weak Isolation
Skills run directly on the host instead of in a sandbox, so they can reach files and credentials they should never touch.

07 AST07: Update Drift
Skills and dependencies change over time; without pinned versions, a safe skill can quietly turn malicious in an update.

08 AST08: Poor Scanning
Weak, pattern-only scanners miss threats hidden in plain-language instructions, letting malicious skills pass review.

09 AST09: No Governance
With no inventory, auditing, or policy enforcement, teams can't see what skills run or respond when one misbehaves.

10 AST10: Cross-Platform Reuse
A skill that's safe on one platform can be dangerous on another when its security settings are lost in the move.

Trust skills like code.
Secure, review, test, and monitor every skill you use.

Verify authenticity | Enforce least privilege | Validate inputs & outputs | Monitor execution | Review dependencies

https://owasp.org/www-project-agentic-skills-top-10/

Ways to Test and Evaluate a Skill?

● Correctness — does the skill work? ● Security — is it safe to run?

AST = OWASP Top 10 for Skills risk it helps defend

01 Trigger testing AST04
Does it activate on the right prompts, and stay quiet on the rest?

02 Eval suites AST08
Score outputs against task scenarios that have known, expected results.

03 Variance analysis AST01
Run the same task many times — one lucky pass isn't reliability.

04 Edge-case testing AST05
Feed empty, huge, or off-topic inputs to find where it breaks.

05 Output validation AST04
Do results match the promised format and quality — not just that it ran?

06 Regression testing AST02 AST07
Re-run the full suite after each edit so fixes don't break old behavior.

07 Static review AST01 AST08
Read the SKILL.md and scripts by hand for hidden or unsafe instructions.

08 Permission audit AST03
Confirm its tools and access match what the task needs — nothing more.

09 Sandbox execution AST06 AST10
Run unfamiliar skills isolated first; watch file, network, and command activity.

10 Adversarial testing AST01 AST05
Probe with malformed inputs and prompt injection to see if it can be hijacked.

Evaluate before you trust
Define success, automate it, repeat for variance, and track every skill over time.

Set success criteria · Automate the checks · Repeat for variance · Probe for abuse · Track over time AST09

Q/A



Thank you!